

Brief Intro and Install Info for jvsipNumpy

Randall Judd

September 18, 2026

Introduction

The VSIPL project is all about a numerical library for signal processing. For python the de facto standard for doing similar calculations is the `textbfnumpy` module. The purpose for writing the **jvsipNumpy** module is to provide a means to enable copies from-and-to **numpy** arrays; and to-and-from **pyJvsip** arrays. The only precisions supported at this time are float and double; both complex and real.

Methods

To try for a little performance we write C code for the various needed copy functions. These are the `numpyArrayCopies.c` and `numpyArrayCopies.h` files. The header file includes needed **python** and **numpy** header files. You must have these headers on your system somewhere. The **setup.py** has code to import the **numpy** header file location from the **numpy** module. The **C vsip** library has copy functions that are needed by the **numpyArrayCopies**. It is expected that the **libvsip.a** has been made and installed in the standard location and **vsip.h** is also in the standard location.

To wrap the code for compiling into a python module we include the **jvsipNumpyUtils.i** file. This file is needed by **SWIG** to generate the **jvsipNumpyUtils_wrap.c** file and the **jvsipNumpyUtils.py** module file.

The **jvsipNumpy.py** file uses the **jvsipNumpyUtils** module and defines two functions. The first is **jvToNp** which takes a **pyJvsip** matrix or vector object and returns the equivalent **numpy** object. The second is **npToJv** which takes a **numpy** array and returns the equivalent **pyJvsip** object.

We assume the file layout is the same as that downloaded with the **JVSIP** package. The **setup.py** file assumes a compiled **vsip** library and header in the default location with respect to the **jvsipNumpy** code. A knowledgeable person could change that by editing these files.

Make and Install

It is recommended you create a virtual python environment and work in that environment when working with the python code in **jvsip**. It is also recommended you install **numpy**, **matplotlib** and **jupyter** for use in the virtual environment.

When in the **jvsipNumpy** directory do

```
python -m pip install .
```

to install **jvsipNumpy** in your virtual environment.

Example

Using a **jupyter notebook** in the virtual environment we demonstrate calculating a singular value decomposition in both **pyJvsip** and **numpy** to ensure we get the same answer with both algorithms.

```
[1]: import numpy as np
import pyJvsip as pj
import jvsipNumpy as jn
f = '%.4f'
np.set_printoptions(precision=4)
```

```
[2]: jvMat = pj.create('mview_d',3,4).randn(5)
npMat = jn.jvToNp(jvMat)
jvMat.mprint(f)
print(npMat)
```

```
[ 0.5080  0.5352  0.6986 -0.9603;
 0.2314  0.0404 -0.4766  0.2077;
 0.5062 -0.3828  0.1575  0.7812]

[[ 0.508  0.5352  0.6986 -0.9603]
 [ 0.2314  0.0404 -0.4766  0.2077]
 [ 0.5062 -0.3828  0.1575  0.7812]]
```

The **mprint** method is designed to print matrices so they can be copied and pasted into Matlab or Octave. Note we just created some data using the normal random number generator in **pyJvsip** with a seed of 5 and then used **jvsipNumpy** to create a **numpy** matrix with the same values. We now do an *SVD* decomposition using **numpy** and **pyJvsip** and compare the results. Basically show they are the same.

```
[3]: u,s,v = jvMat.svd
U,S,VT = np.linalg.svd(npMat)
```

We note that the **V** matrix is returned in **pyJvsip** (just the way I did it) and **V Transpose** is returned by **numpy**.

```
[4]: u.mprint(f)
print(U)

[ 0.8783 -0.4374 -0.1928;
 -0.2120  0.0051 -0.9773;
 -0.4285 -0.8992  0.0882]

[[ 0.8783 -0.4374 -0.1928]
 [-0.212  0.0051 -0.9773]
 [-0.4285 -0.8992  0.0882]]
```

```
[5]: s.mprint(f)
print(S)
```

```
[ 1.5285  0.8665  0.4805]
```

```
[1.5285 0.8665 0.4805]
```

```
[6]: v.trans.mprint(f)
      print(VT)
```

```
[ 0.1179  0.4092  0.4234 -0.7996;
 -0.7804  0.1274 -0.5189 -0.3247;
 -0.5816 -0.3673  0.7180  0.1065;
 -0.1968  0.8255  0.1896  0.4939]
```

```
[[ 0.1179  0.4092  0.4234 -0.7996]
 [-0.7804  0.1274 -0.5189 -0.3247]
 [-0.5816 -0.3673  0.718  0.1065]
 [-0.1968  0.8255  0.1896  0.4939]]
```

Inspection shows we have equivalent results for numpy and pyJvsip.

```
[ ]:
```